

Unit 1

Data Structure

A data structure is a way of organizing and storing data so that it can be used efficiently. It helps in performing operations like insertion, deletion, searching, and sorting in a structured way.

Types of Data Structures:

1. **Array** – Stores elements in a fixed-size list.
2. **Linked List** – A chain of connected elements.
3. **Stack** – Follows LIFO (Last In, First Out) rule.
4. **Queue** – Follows FIFO (First In, First Out) rule.
5. **Tree** – A hierarchical structure like a family tree.
6. **Graph** – Represents connections like a social network.
7. **Hash Table** – Stores key-value pairs for quick lookups.

Algorithms in Data Structures: Algorithms in data structures are step-by-step methods used to perform operations like searching, sorting, inserting, deleting, and traversing data efficiently.

Types of Algorithms in Data Structures:

1. **Searching Algorithms:** Used to find an element in a data structure.
 - **Linear Search** – Checks each element one by one ($O(n)$).
 - **Binary Search** – Divides the array and searches efficiently ($O(\log n)$, only for sorted arrays).
2. **Sorting Algorithms:** Used to arrange elements in ascending or descending order.
 - **Bubble Sort** – Swaps adjacent elements until sorted ($O(n^2)$).
 - **Selection Sort** – Finds the smallest element and places it at the beginning ($O(n^2)$).
 - **Insertion Sort** – Inserts elements in their correct position ($O(n^2)$).
 - **Merge Sort** – Uses the divide-and-conquer method ($O(n \log n)$).
 - **Quick Sort** – Selects a pivot and sorts partitions ($O(n \log n)$ best case).
3. **Insertion Algorithms:** Used to insert an element into a data structure efficiently.
 - **Array Insertion** – Adds an element at a specific index.
 - **Linked List Insertion** – Adds a node at the beginning, middle, or end.
4. **Deletion Algorithms:** Used to remove elements from a data structure.
 - **Array Deletion** – Removes an element and shifts remaining elements.
 - **Linked List Deletion** – Removes a node by updating pointers.

5. **Traversal Algorithms:** Used to visit each element in a data structure.
 - **Array Traversal** – Accesses each element one by one.
 - **Tree Traversal** – Visits nodes using Preorder, Inorder, and Postorder methods.
 - **Graph Traversal** – Uses BFS (Breadth-First Search) and DFS (Depth-First Search).
6. **Hashing Algorithms:** Used for fast data retrieval using key-value pairs.
 - **Direct Hashing** – Stores values using direct key mapping.
 - **Collision Handling (Chaining, Open Addressing)** – Manages multiple elements at the same position.

Array: An array is a fundamental data structure used to store a collection of elements, typically of the same data type, in contiguous memory locations. It allows efficient access to elements using an index.

Characteristics of an Array

1. **Fixed Size** – The size of an array is defined at the time of declaration and cannot be changed dynamically.
2. **Indexed Access** – Elements are stored sequentially and accessed using an index (starting from 0 in most programming languages).
3. **Same Data Type** – All elements in an array must be of the same data type.
4. **Contiguous Memory Allocation** – Elements are stored in adjacent memory locations, making retrieval efficient.

Linear Search: Linear Search is a simple searching algorithm that checks each element in a list one by one until the desired element is found or the list ends.

Binary Search: Binary Search is an efficient searching algorithm used on sorted arrays. It works by repeatedly dividing the array into halves until the target element is found.

Linked List: A linked list is a linear data structure where elements, called nodes, are connected using pointers. Unlike arrays, linked lists do not have a fixed size and can dynamically grow or shrink as needed.

Memory Allocation: Memory allocation in data structures refers to how memory is assigned to store variables, arrays, and objects during program execution. It is crucial for efficient memory management and performance.

Types of Memory Allocation

1. **Static Memory Allocation (Compile-Time Allocation):** Memory is allocated before execution (at compile-time). The size of memory cannot change during runtime. Faster but may lead to memory wastage if the allocated space is too large.
2. **Dynamic Memory Allocation (Run-Time Allocation):** Memory is allocated during execution (runtime). The size of memory can grow or shrink as needed. Requires manual deallocation to prevent memory leaks.

Garbage Collection: Garbage Collection (GC) is the process of automatically reclaiming unused memory occupied by objects or variables that are no longer needed. This prevents **memory leaks** and optimizes memory usage.

Linear Search(Array A, Value X) Algorithm

Step1: Set i to 1

Step2: if $i > n$ then go to step 7

Step3: if $A[i] = x$ then go to step 6

Step4: Set i to $i + 1$

Step5: Go to step 2

Step6: Print Element x found at index i and go to step 8

Step7: Print element not found

Step8: Exit

Explanation: {2, 5, 8, 6, 23, 38, 56, 72, 91}

$$0+9/2 = 4.5$$

23= 5 location

Binary Search

Step1: Set beg = lower_bound, end = upper_bound, pos

Step2: Repeat steps3 and 4 while $beg \leq end$

Step3: Set $mid = (beg + end)/2$

- if $a[mid] = val$
- Set pos
- Go to step 6
- Else if $a[mid] > val$
- Set $end = mid - 1$
- Else
- Set $beg : mid + 1$
- [end of if] and [end of loop]

Step4: if $pos = -1$

Step5: Print “Value is not present in the array”

- [end of if]

Step6: Exit

Insertion in an Array

INSERT (A, N, K, ITEM)

Step1: Set $J = N$

Step2: Repeat step 3 and 4 while $J \geq K$

Step3: Set $A[J+1] = A[J]$

Step4: Set $J = J-1$

- [End of step 2 loop]

Step5: $A[K] = \text{ITEM}$

Step6: Set $N = N+1$

Step7: END

Deletion from an Array

DELETE (A, N, K, ITEM)

Step1: Set $\text{ITEM} = A[K]$

Step2: Repeat for $J = K$ to $N-1$

- Set $A[J] = A[J+1]$
- [End of loop]

Step3: Set $N = N-1$

Step4: Exit

Bubble Sort: Bubble Sort is the simplest sorting algorithm that works by repeatedly swapping the adjacent elements if they are in the wrong order. This algorithm is not suitable for large data sets as its average and worst-case time complexity are quite high.

Bubble Sort of an Array

Bubble (DATA, N)

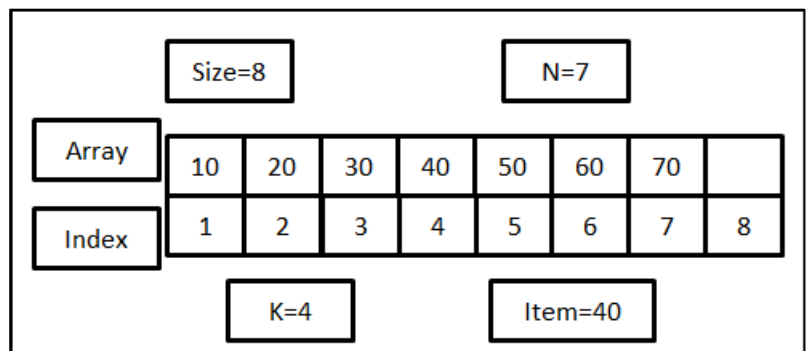
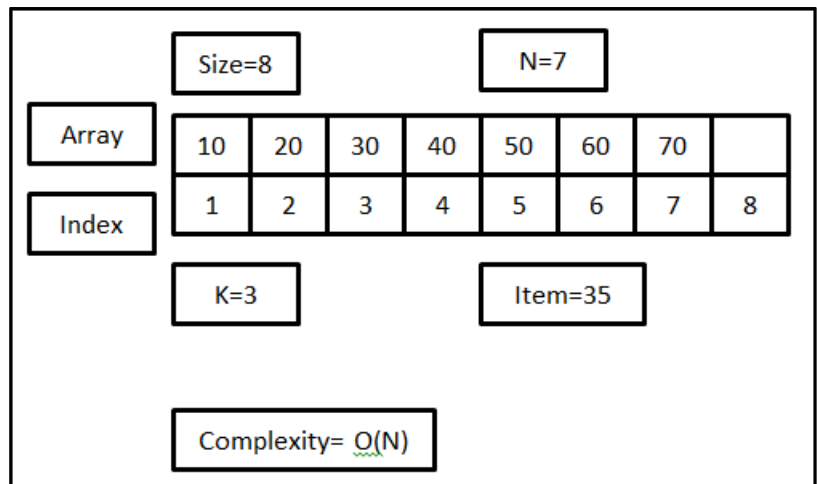
Here DATA is an array with N elements. This algorithm sort the element in DATA.

Step1: Repeat steps 2 and 3 for $K = 1$ to $N - 1$

Step2: Set $\text{PTR} = 1$ [Initialize pass pointer PTR]

Step3: Repeat while $\text{PTR} \leq N-K$ [Execute PASS]

- (a) If $\text{DATA}[\text{PTR}] > \text{DATA}[\text{PTR} + 1]$
 - Interchange $\text{DATA}[\text{PTR}]$ & $\text{DATA}[\text{PTR} + 1]$
 - [End of IF structure]
- (b) Set $\text{PTR} = \text{PTR} + 1$



- [End of Inner loop]
- [End of step 1 outer loop]

Step4: Exit

Ques: 32 $\longleftrightarrow$ 51 27 85 66 23 13 57

Ans: 32 51 $\longleftrightarrow$ 27 87 66 23 13 57

1 Pass 32 27 51 $\longleftrightarrow$ 85 66 23 13 57

32 27 51 85 $\longleftrightarrow$ 66 23 13 57

32 27 51 66 85 $\longleftrightarrow$ 23 13 57

32 27 51 66 23 85 $\longleftrightarrow$ 13 57

32 27 51 66 23 13 85 $\longleftrightarrow$ 57

32 27 51 66 23 13 57 85

2 Pass 32 $\longleftrightarrow$ 27 51 66 23 13 57 58

27 32 $\longleftrightarrow$ 51 66 23 13 57 58

27 32 51 $\longleftrightarrow$ 66 23 13 57 58

27 32 51 66 $\longleftrightarrow$ 23 13 57 58

27 32 51 23 66 $\longleftrightarrow$ 13 57 58

27 32 51 23 13 66 $\longleftrightarrow$ 57 58

27 32 51 23 13 57 66 $\longleftrightarrow$ 58

27 32 51 23 13 57 58 66

3 Pass 27 $\longleftrightarrow$ 32 51 23 13 57 58 66

27 32 $\longleftrightarrow$ 51 23 13 57 58 66

27 32 51 $\longleftrightarrow$ 23 13 57 58 66

27 32 23 51 $\longleftrightarrow$ 13 57 58 66

27 32 23 13 51 $\longleftrightarrow$ 57 58 66

27 32 23 13 51 57 $\longleftrightarrow$ 58 66

27 32 23 13 51 57 58 $\longleftrightarrow$ 66

27 32 23 13 51 57 58 66

4 Pass 27 $\longleftrightarrow$ 32 23 13 51 57 58 66

27 32 $\longleftrightarrow$ 23 13 51 57 58 66

	27	23	32	13	51	57	58	66
	27	23	13	32	51	57	58	66
	27	23	13	32	51	57	58	66
	27	23	13	32	51	57	58	66
	27	23	13	32	51	57	58	66
	27	23	13	32	51	57	58	66
5 Pass	27	23	13	32	51	57	58	66
	23	27	13	32	51	57	58	66
	23	13	27	32	51	57	58	66
	23	13	27	32	51	57	58	66
	23	13	27	32	51	57	58	66
	23	13	27	32	51	57	58	66
	23	13	27	32	51	57	58	66
	23	13	27	32	51	57	58	66
6 Pass	23	13	27	32	51	57	58	66
	13	23	27	32	51	57	58	66
	13	23	27	32	51	57	58	66
	13	23	27	32	51	57	58	66
	13	23	27	32	51	57	58	66
	13	23	27	32	51	57	58	66
F. Ans	13	23	27	32	51	57	58	66

Linked List(Traversing)

Step1: Set PTR = START(Initilize pointer PTR)

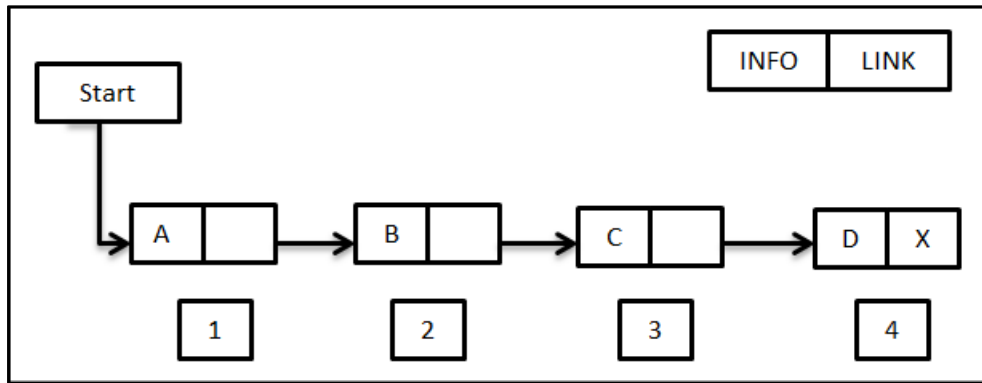
Step2: Repeat step 3 and 4 while PTR $\neq$ NULL

Step3: Apply PROCESS to INFO [PTR]

Step4: Set PTR = LINK [PTR] [PTR now points to the next node].

[End of step 2 loop]

Step5: Exit



Linked List Unsorted(Searching)

SEARCH (INFO, LINK, START, ITEM, LOC)

Step1: Set PTR = START

Step2: Repeat step 3 while PTR $\neq$ NULL

Step3: if ITEM = INFO [PTR], then

- Set LOC = PTR & Exit
- Else:
- Set PTR = LINK [PTR]
- [Now PTR points to next nodes]
- [End of step 2 loop]

Step4: [Search Unsuccessful?] LOC = NULL

Step5: Exit

Linked List Sorted(Searching)

SEARCH (INFO, LINK, START, ITEM, LOC)

Step1: Set PTR = START

Step2: Repeat step 3 while PTR $\neq$ NULL

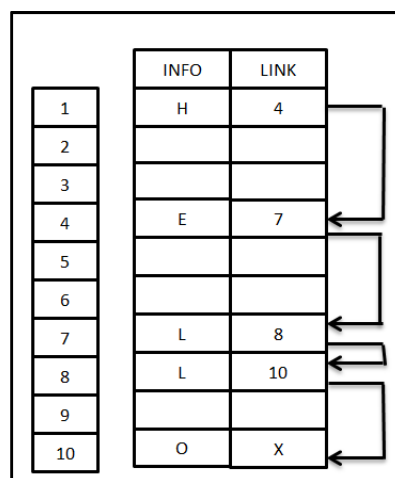
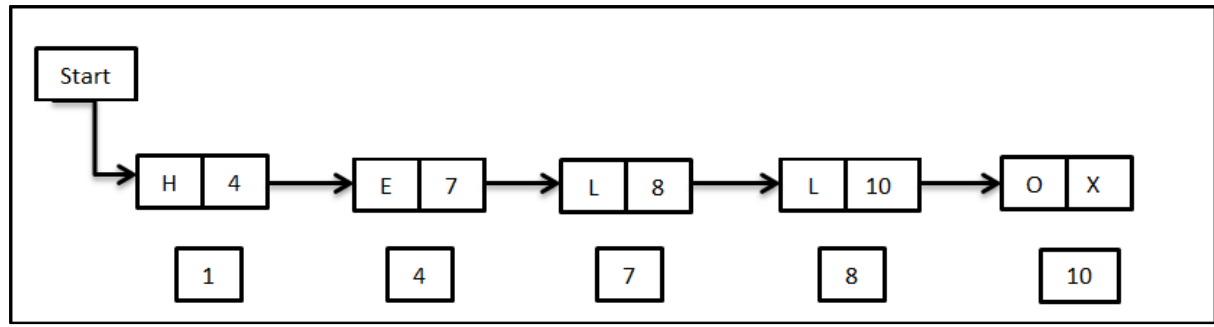
Step3: if ITEM > INFO [PTR], then

- Set PTR = LINK[PTR] [PTR points to the next node]
- Elseif ITEM = INFO[PTR] then
- Set LOC = PTR, & Exit
- Else:
- Set LOC = NULL & Exit
- [End of if structure]
- [End of step 2 loop]

Step4: Set LOC = NULL

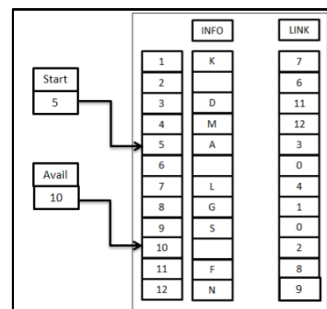
Step5: Exit

Memory representation of a Linked List (means how to data store in a linked list)



Memory allocation

LIST (INFO, LINK, START, AVAIL)



(a) Insert into a Linked List (Begging)

INS FIRST (INFO, LINK, START, AVAIL, ITEM)

This algorithm inserts ITEM as the first node in the list

Step1: [OVERFLOW?] if AVAIL = NULL, then Write: OVERFLOW & Exit

Step2: [Remove first node from AVAIL LIST]

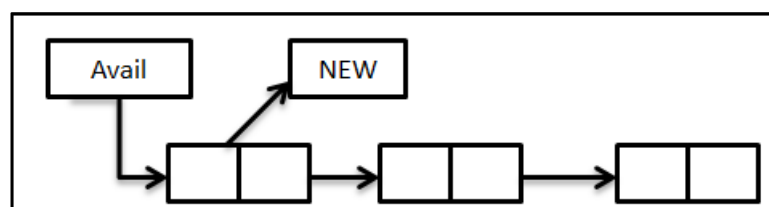
- Set NEW = AVAIL = LINK [NEW]

Step3: Set INFO[NEW] = ITEM [Copies new data into new node]

Step4: Set LINK [NEW] = START [NEW node now points to original first node]

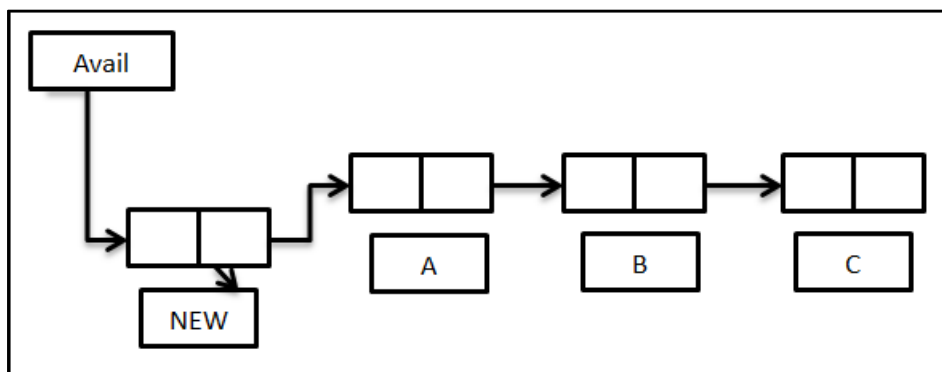
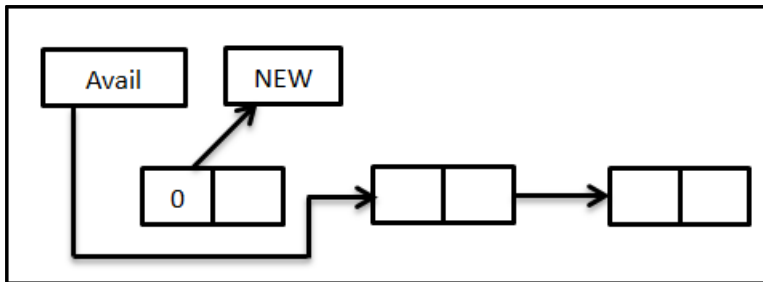
Step5: Set START = NEW [Changes START so that it points to the new node]

Step6: Exit



NEW = AVAIL

AVAIL = LINK [NEW]



LINK [NEW] = START

START = NEW

(a) Insertion after a given location

INS LOC [INFO, LINK, START, ITEM, LOC]

This algorithm inserts ITEM so that ITEM follow the node with location(LOC) or inserts ITEM as the first node when LOC = NULL

Step1: [OVERFLOW?] if AVAIL = NULL, then Write: OVERFLOW and Exit

Step2: [Remove first node from AVAIL LIST]

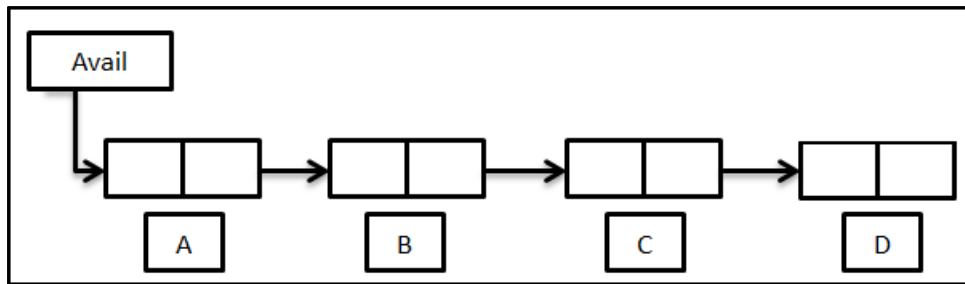
- Set NEW = AVAIL & AVAIL = LINK [NEW]

Step3: Set INFO [NEW] = ITEM [Copies new data into new node]

Step4: if LOC = NULL, then [inserts as first node]

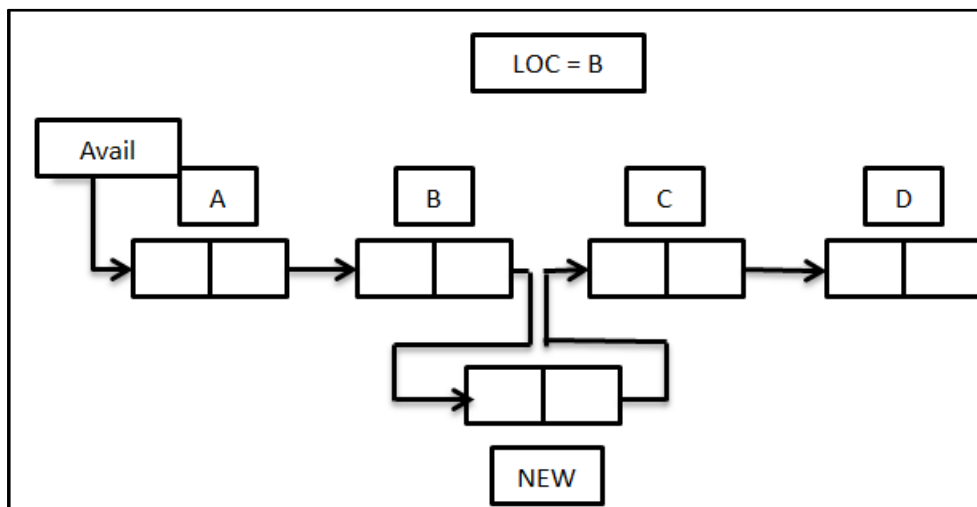
- Set LINK [NEW] = START & START = NEW
- ELSE: [INSERT after node with location LOC]
- Set LINK [NEW] = LINK [LOC] and LINK[LOC] = NEW
- [End of if structure]

Step5: Exit



NEW = AVAIL

AVAIL = LINK [NEW]



(c) Deletion from Linked List

DEL (INFO, LINK, START, AVAIL, LOC, LOCP)

This algorithm deletes the node N with location LOC. LOCP is the node which proceeds nor where N is the first node, LOCP = NULL

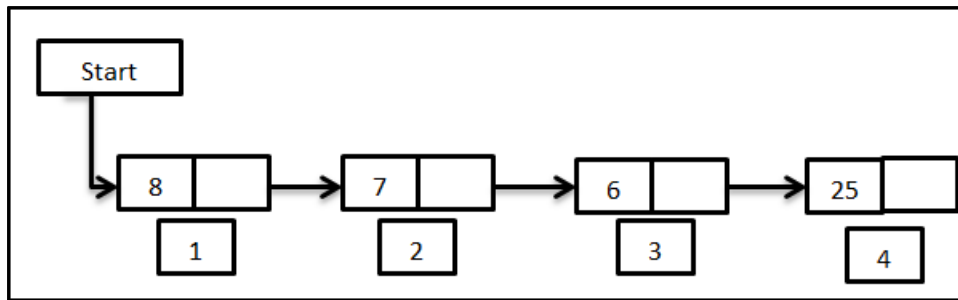
Step1: if LOCP = NULL, then:

- Set START = LINK [START]
- [Delete first node]
- Else:
- Set LINK [LOCP] = LINK [LOC]
- [Delete node N]
- [End of if structure]

Step2: [Return delete node in Avail]

- Set LINK [LOC] = AVAIL
- Set AVAIL = LOC

Step3: Exit

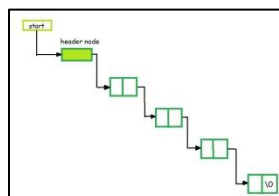


Note: LOCP was previous node. LOC is next node of the previous one.

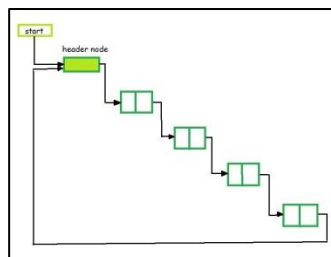
Types of Linked List

1. **Header Linked List:** A header linked list is a special type of linked list which consists of a **header** node in the **beginning** of the list. The header node can store **meta data** about the linked list. This type of list is useful when information other than that found in each node is needed. For example, suppose there is an application in which the **number of items** in a list is often **calculated**. Usually, a list is always traversed to **find the length** of the list. However, information can be easily obtained if the **current length** is maintained in an additional header node.

- (a) **Grounded Header Linked List:** A grounded header linked list is a linked list where the last node points to null. It is a type of header-linked list with a special node at the beginning, called the header node. The header node allows access to all nodes in the list and does not need to represent the same data type as other nodes. A grounded linked list is a type of linked list where the last node points to null. It has a special node at the beginning called the header node.

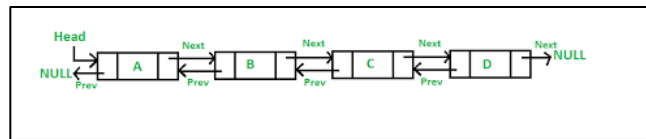


- (b) **Circular Header Linked List:** A "Circular Header Linked List" is a type of linked list where the last node in the list points back to a special "header node" at the beginning, effectively creating a circular structure where traversing through the list will always loop back to the header node; essentially combining the features of a circular linked list and a header linked list.



2. **Doubly Linked List:** A doubly linked list is a more complex data structure than a singly linked list, but it offers several advantages. The main advantage of a doubly linked list is that it allows for efficient traversal of the list in both directions. This is because each node in the list contains a pointer to the previous node and a pointer to the next node. This allows

for quick and easy insertion and deletion of nodes from the list, as well as efficient traversal of the list in both directions.

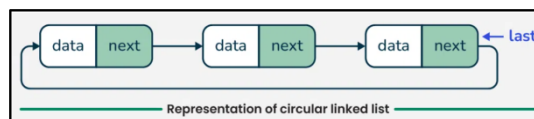


3. Circular

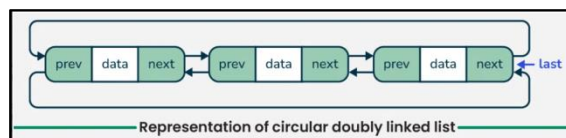
Linked List: A

circular linked list is a special type of linked list where all the nodes are connected to form a circle. Unlike a regular linked list, which ends with a node pointing to NULL, the last node in a circular linked list points back to the first node. This means that you can keep traversing the list without ever reaching a NULL value.

- (a) **Circular Singly Linked List:** In Circular Singly Linked List, each node has just one pointer called the “next” pointer. The next pointer of the last node points back to the first node and this results in forming a circle. In this type of Linked list, we can only move through the list in one direction.



- (b) **Circular Doubly Linked List:** In circular doubly linked list, each node has two pointers prev and next, similar to doubly linked list. The prev pointer points to the previous node and the next points to the next node. Here, in addition to the last node storing the address of the first node, the first node will also store the address of the last node.



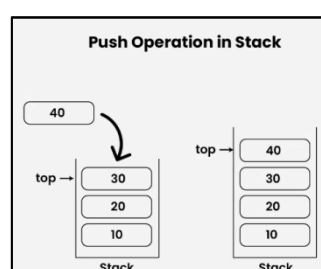
Stack: A **Stack** is a linear data structure that follows a particular order in which the operations are performed. The order may be **LIFO(Last In First Out)** or **FILO(First In Last Out)**. **LIFO** implies that the element that is inserted last, comes out first and **FILO** implies that the element that is inserted first, comes out last.

Push Operation on Stack

Adds an item to the stack. If the stack is full, then it is said to be an **Overflow condition**.

Algorithm for Push Operation:

- Before pushing the element to the stack, we check if the stack is **full**.
- If the stack is full (**top == capacity-1**), then **Stack Overflows** and we cannot insert the element to the stack.
- Otherwise, we increment the value of top by 1 (**top = top + 1**) and the new value is inserted at **top position**.
- The elements can be pushed into the stack till we reach the **capacity** of the stack.

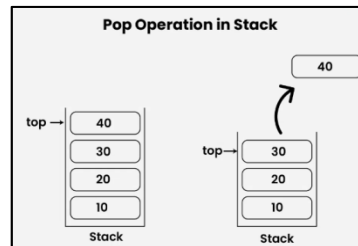


Pop Operation in Stack

Removes an item from the stack. The items are popped in the reversed order in which they are pushed. If the stack is empty, then it is said to be an **Underflow condition**.

Algorithm for Pop Operation:

- Before popping the element from the stack, we check if the stack is **empty**.
- If the stack is empty ($\text{top} == -1$), then **Stack Underflows** and we cannot remove any element from the stack.
- Otherwise, we store the value at top, decrement the value of top by 1 ($\text{top} = \text{top} - 1$) and return the stored top value.



Array representation of Stack

(PUSH)

Step1: [Check for stack overflow] if $\text{TOP} \geq \text{MAXSTACK}$ print "Stack overflow" & exit.

Step2: [increment the pointer value by one] $\text{TOP} = \text{TOP} + 1$

Step3: [Insert the item] over $[\text{TOP}] = \text{value}$

Step4: Exit

(POP)

Step1: [Check whether the stack is empty] if $\text{TOPs} = 0$ print "Stack underflow" & exit

Step2: [Remove the top most item] $\text{value} = \text{arr}[\text{TOP}]$ $\text{TOP} = \text{TOP} - 1$

Step3: [Return the item of the stack] return (value)

Link representation of stack

(PUSH)

PUSH (INFO, LINK, TOP, ITEM, AVAIL)

This algorithm push an element to the stack

Step1: if $\text{AVAIL} == \text{NULL}$. Then

Write "OVERFLOW"

Return

Step2: Set $\text{NEW} = \text{AVAIL}$ and $\text{AVAIL} = \text{AVAIL} \rightarrow \text{LINK}$

Step3: Set NEW -> INFO = ITEM

Step4: Set NEW -> LINK = TOP

- Set TOP = NEW

Step5: Exit

(POP)

Step1: if (TOP is equal to NULL)

(a)Display “The stack is empty”

Step2: else

(a)TEMP = TOP

(b)Display “The popped element TOP -> DATA”

(c)TOP = TOP -> Next

(d)TEMP -> Next – NULL

(e)Free the TEMP node

Step3: Exit

Applications of Stack

1. Evaluation of postfix notation
2. Transforming infix to postfix notation

Polish notation

A+B (infix)

-AB (prefix)

AB+ (postfix)

(a) Evaluation of postfix notation

Algorithm

This algorithm finds the VALUE of an arithmetic expression P written in postfix notation.

Step1: Add a right parenthesis”)” t the end of P. [This acts as a sentinel]

Step2: Scan P from left to right and repeat Steps3 and 4 foe each element of P until the sentinel “)” is encountered.

Step3: if an operand is encountered, put it on STACK

Step4: if an operator * is encountered, then:

- Remove the two top elements of STACK, where A is the top element B is the next to top element
- Evaluate B * A
- Place the result of (b) back on STACK
- [End of if structure]

Step5: Set VALUE equal to the top element on STACK

Step6: Exit

Ex: Evaluation the following postfix notation: BTS : 5, 6, 2, +, *, 12, 4, /, -

Solution: BTS: 5, 6, 2, +, *, 12, 4, /, -)

	Symbol Scanned	Stack
1	5	5
2	6	5, 6
3	2	5, 6, 2
4	+	5, 8
5	*	40
6	12	40, 12
7	4	40, 12, 4
8	/	40, 3
9	-	37
10	)	—

(b) Transforming infix to postfix notation

Algorithm: POLISH(Q, P)

Suppose Q is an arithmetic expression written in infix notation. This algorithm finds the equivalent postfix expression P.

Step1: Push “(“ onto STACK, AND ADD “)” to the end of Q.

Step2: Scan Q from left to right and repeat Steps3 and 6 for each element of Q until the STACK is empty:

Step3: if an operand is encountered, add it to P.

Step4: if a left parenthesis is encountered, push it onto STACK

Step5: if an operator * is encountered, then:

- Repeatedly pop from STACK and to P each operator (on the top of STACK) which has the same precedence as or higher precedence than *.
- Add * to STACK.
- [End of if structure]

Step6: if a right parenthesis is encountered, then:

- Repeatedly pop from STACK and add to P each operator (on the top of STACK) until a left parenthesis is encountered.
- Remove the left parenthesis.[Do not add the left parenthesis to P.]
- [End of if structure]
- [End of step 2 loop]

Step7: Exit

Question: $A + (B * C - (D / E \uparrow F) * G) * H$

Solution: $A + (B * C - (D / E \uparrow F) * G) * H$

	Symbol Scanned	Stack	Expression P
1	A	(	A
2	+	(+	A
3	(	(+(	A
4	B	(+(	A
5	*	(+(*	AB
6	C	(+(*	ABC
7	-	(+(-	ABC*
8	(	(+(-(	ABC*
9	D	(+(-(	ABC*D
10	/	(+(-(/	ABC*D
11	E	(+(-(/	ABC*DE
12	$\uparrow$	(+(-(/ $\uparrow$	ABC*DE
13	F	(+(-	ABC*DEF $\uparrow$ /
14	)	(+(-*	ABC*DEF $\uparrow$ /
15	*	(+(-	ABC*DEF $\uparrow$ /
16	G	(+(-*	ABC*DEF $\uparrow$ /G*-
17	)	(+	ABC*DEF $\uparrow$ /G*-
18	*	(+*	ABC*DEF $\uparrow$ /G*-
19	H	(+*	ABC*DEF $\uparrow$ /G*-H
20	)	-	ABC*DEF $\uparrow$ /G*-H*+

Quick Sort: Quick Sort is a sorting algorithm based on the [Divide and Conquer](#) that picks an element as a pivot and partitions the given array around the picked pivot by placing the pivot in its correct position in the sorted array.

Ques: 44 33 11 55 77 90 40 60 99 22 88 66
Ans: 22 33 11 55 77 90 40 60 99 44 88 66
 22 33 11 44 77 90 40 60 99 55 88 66
 22 33 11 40 77 90 44 60 99 55 88 66
 22 33 11 40 44 90 77 60 99 55 88 66

1 Sub. 22 33 11 40
 11 33 22 40

F. Ans: 11 22 33 40

2 Sub. 90 77 99 55 88 66
 66 77 99 55 88 90
 66 77 90 55 88 99
 66 77 88 55 90 99

3 Sub. 66 77 88 55
 55 77 88 66
 55 66 88 77

4 Sub. 88 77
 77 88

How does QuickSort Algorithm work?

QuickSort works on the principle of **divide and conquer**, breaking down the problem into smaller sub-problems.

There are mainly three steps in the algorithm:

1. **Choose a Pivot:** Select an element from the array as the pivot. The choice of pivot can vary (e.g., first element, last element, random element, or median).
2. **Partition the Array:** Rearrange the array around the pivot. After partitioning, all elements smaller than the pivot will be on its left, and all elements greater than the pivot will be on its right. The pivot is then in its correct position, and we obtain the index of the pivot.
3. **Recursively Call:** Recursively apply the same process to the two partitioned sub-arrays (left and right of the pivot).
4. **Base Case:** The recursion stops when there is only one element left in the sub-array, as a single element is already sorted.

Difference between push and pop in stack

Feature	Push (Insertion)	Pop (Deletion)
Definition	Adds an element to the stack.	Removes the top element from the stack.
Condition	Stack Overflow (if full).	Stack Underflow (if empty).
Effect	Increases stack size.	Decreases stack size.
Example	<code>Push(10), Push(20) → [10, 20]</code>	<code>Pop() → [10], Pop() → []</code>
Time Complexity	O(1)	O(1)

Difference between stack and queue

Feature	Stack 📦	Queue 🚶
Definition	A data structure that follows LIFO (Last In, First Out).	A data structure that follows FIFO (First In, First Out).
Insertion (Push/Enqueue)	Elements are pushed to the top.	Elements are added to the rear (back).
Deletion (Pop/Dequeue)	Elements are popped from the top.	Elements are removed from the front.
Order of Execution	Last element inserted is the first to be removed.	First element inserted is the first to be removed.
Operations	Push (insert), Pop (remove), Peek (top element).	Enqueue (insert), Dequeue (remove), Front/Rear (view elements).
Usage	Used for backtracking, recursion, undo operations.	Used in scheduling, queues, real-time processing.
Types	Simple Stack, Linked Stack.	Simple Queue, Circular Queue, Double-ended Queue (Deque), Priority Queue.
Time Complexity	O(1) for Push & Pop. ⬇	O(1) for Enqueue & Dequeue.

Operations of Queue

1. Enque (insertion of queue)
2. Deque (deletion of queue)

Enque (insertion of queue)

Step1: If $REAR = MAX - 1$

- Write “Queue is overflow”
- Go to step4
- [End of if]

Step2: if FRONT = -1 and REAR = -1

- Set FRONT = REAR = 0
- Else
- Set REAR = REAR + 1
- [End of if]

Step3: Set Queue [REAR] = NUM

Step4: Exit

Deque (deletion of queue)

Step1: if FRONT = -1 or FRONT > REAR

- Write “Queue is Underflow”
- Else
- Set val = Queue [FRONT]
- FRONT = FRONT + 1
- [End of if]

Step2: Exit

Types of Queue

1. Circular Queue
2. Priority Queue
 - Ascending Priority Queue
 - Descending Priority Queue
3. Double Ended Queue
 - Input Restricted Deque
 - Output Restricted Deque

Array Representation of Queue

- (a) Initially Empty
FRONT = 0 and REAR = 0
- (b) A, B, C are inserted
FRONT = 1 and REAR = 3
- (c) A deleted
FRONT = 2 and REAR = 3
- (d) D, E inserted
FRONT = 2 and REAR = 5

A	B	C		
1	2	3	4	5
	B	C		
1	2	3	4	5

	B	C	D	E
1	2	3	4	5
			D	E
1	2	3	4	5
F			D	E
1	2	3	4	5

- (e) B and C deleted
FRONT = 4 and REAR = 5

F				E
1	2	3	4	5

F	G	H		E
1	2	3	4	5

F	G	H		
1	2	3	4	5

- (f) F inserted
FRONT = 4 and REAR = 1

- (g) D delete
FRONT = 5 and REAR = 1

- (h) G and H inserted
FRONT = 5 and REAR = 3

- (i) E deleted
FRONT = 1 and REAR = 3

- (j) F deleted
FRONT = 2 and REAR = 3

- (k) K inserted
FRONT = 2 and REAR = 4

- (l) G and H deleted
FRONT = 4 and REAR = 4

	G	H		
1	2	3	4	5

	G	H	K	
1	2	3	4	5

			K	
1	2	3	4	5

1	2	3	4	5

- (m) K deleted and Queue empty
FRONT = 0 and REAR = 0

Important: Degree of a Node: In data structures—especially **tree structures** like **binary trees**, **general trees**, or **graphs**—the **degree of a node** refers to the **number of direct connections** or **edges** it has. The degree of a node is the number of children of that node. The degree of a tree is the maximum degree of a nodes in given node.

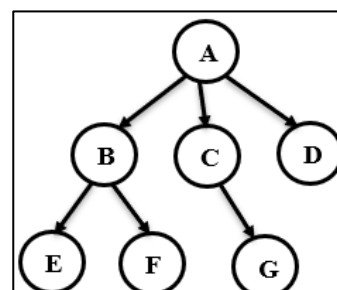
Path: It is the sequence of consecutive edges from source node to destination node.

Height of a node: The height of a node is the maximum path length from that node to a leaf node.

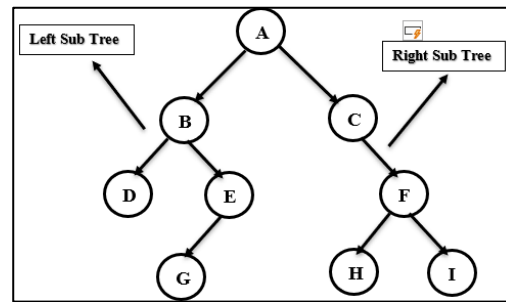
Height of a tree: The height of a tree is the height of the root.

Depth of a tree: The **depth of a tree** is the **maximum depth among all nodes**.

1. A is the root node.
2. B is the parent of E and F.



3. D is the sibling of B and C.
4. E and F are children of B.
5. E, F, G and D are external nodes or leaves.
6. A, B and C are internal nodes.
7. Depth of F is 2.
8. The height of tree is 2.
9. The degree of node A is 3.
10. The degree of tree is 3.



A **Binary Tree** is a **non-linear hierarchical data structure** in which **each node has at most two children**, commonly referred to as:

- **Left child**
- **Right child**
- Each node contains:
 - A data element
 - A pointer to the left child
 - A pointer to the right child
- The topmost node is called the **root**.
- Nodes with no children are called **leaf nodes**.
- The **maximum number of nodes** at level L is 2^L .

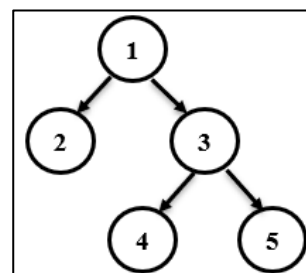
Strictly Binary Tree in Data Structure

A **Strictly Binary Tree** (also called a **Proper Binary Tree**) is a type of binary tree where **each internal node has exactly two children**, and **leaf nodes have zero children**.

- Every node has **either 0 or 2 children**.
- No node in the tree has only **one child**.
- The tree can be **incomplete** (not all leaves are at the same level), but the "strict" rule on number of children applies.

Example of a Strictly Binary Tree:

- Node 1 $\rightarrow$ 2 children
- Node 2 $\rightarrow$ 0 children
- Node 3 $\rightarrow$ 2 children
- Nodes 4 and 5 $\rightarrow$ 0 children

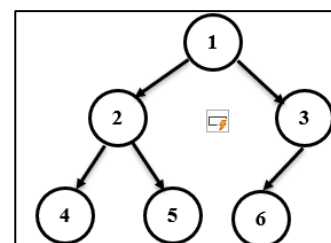


Complete Binary Tree in Data Structure

- A **Complete Binary Tree** is a special type of **binary tree** where:
- **All levels are completely filled** except possibly the **last level**,
- And all nodes in the last level are **as far left as possible**.
- Every level (except possibly the last) has **maximum number of nodes**.
- The last level is filled from **left to right**.
- It is used in many applications like **heaps** (e.g., binary heap for priority queues).

Example of a Complete Binary Tree

- All levels except the last are **fully filled**.



- The last level is filled from **left to right**.

Almost Complete Binary Tree (ACBT) in Data Structure

An **Almost Complete Binary Tree** is a **binary tree** that satisfies the following conditions:

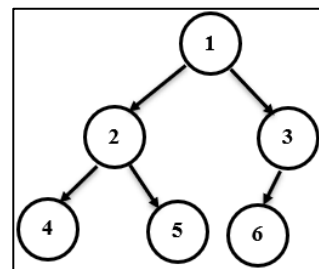
Definition:

A **binary tree** is called an **Almost Complete Binary Tree** if:

1. All **levels** are **completely filled**, except possibly the **last level**.
2. In the **last level**, all **nodes** are as far left as possible (i.e., they fill from left to right without gaps).

Example:

- Level 0: 1
- Level 1: 2, 3
- Level 2: 4, 5, 6 (no gaps; left to right filled)
- This is an **almost complete** (and also **complete**) binary tree.



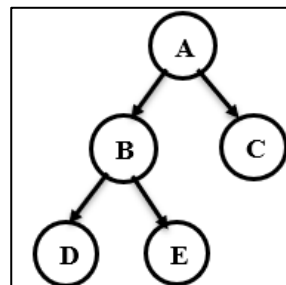
Operations of a Binary Tree: Preorder, Inorder, and Postorder Traversals

In a binary tree, **tree traversal** refers to the process of visiting each node **exactly once** in a specific order. The three primary **depth-first** traversal techniques are:

Inorder Traversal (Left – Root – Right)

- Traverse the **left subtree**
- Visit the **root node**
- Traverse the **right subtree**

Inorder: D B E A C



Preorder Traversal (Root – Left – Right)

- Visit the **root node**
- Traverse the **left subtree**
- Traverse the **right subtree**

Preorder: A B D E C

Postorder Traversal (Left – Right – Root)

- Traverse the **left subtree**
- Traverse the **right subtree**
- Visit the **root node**

Postorder: D E B C A

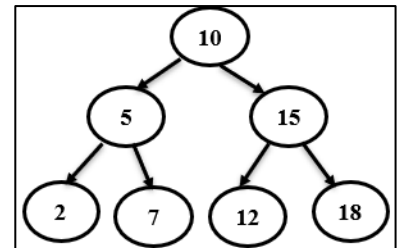
Balanced Binary Search Tree (Balanced BST) in Data Structure

A **Balanced Binary Search Tree (BST)** is a special type of **binary tree** where:

- **Left subtree** contains values **less than** the node.
- **Right subtree** contains values **greater than** the node.
- The **height difference** (or depth) between the **left and right subtrees** of **every node** is at most **1**.

Why Balanced BST?

- A regular BST can become **unbalanced**, resembling a **linked list**, leading to **$O(n)$** time operations.
- A **Balanced BST** ensures:
 - **Search, Insert, Delete** = **$O(\log n)$** in worst case.



Example of a Balanced BST:

- Each node's left and right subtrees have **heights that differ by no more than 1**.

Red-Black Tree (Balanced Binary Search Tree): A **Red-Black Tree** is a type of **self-balancing Binary Search Tree (BST)** where each node has an extra bit to store **color information** (either Red or Black). It ensures the tree remains approximately balanced, leading to **$O(\log n)$** time complexity for insertion, deletion, and search.

Key Properties of a Red-Black Tree

1. Every node is either red or black.
2. The root is always black.
3. All leaves (NULL children) are black.
4. If a node is red, then both its children must be black (no two red nodes can be adjacent).
5. Every path from a node to its descendant NULL nodes must have the same number of black nodes (called black-height).

Why Red-Black Tree?

- To maintain a **balanced tree** after **insertions** and **deletions**, without requiring full tree re-balancing.
- Guarantees a tree height of at most **$2 \times \log(n)$** → so operations remain efficient.

Operation	Time Complexity	Notes
Search	$O(\log n)$	Same as BST
Insertion	$O(\log n)$	May require rotations and recoloring
Deletion	$O(\log n)$	More complex than insertion

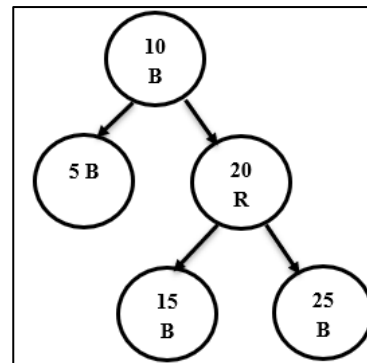
Rotations

- Used to rebalance the tree.
- Types:
 - **Left Rotation**

- **Right Rotation**
- (Sometimes combinations of both)

Example Structure

- Root is black.
- Red nodes (5, 20) have black children.
- Every path from root to NULL leaf has same number of black nodes.



Advantages

- Balanced search time.
- Ensures no skewed trees like in unbalanced BSTs.

Disadvantages

- More complex implementation than AVL Trees.
- Slightly slower insertion/deletion than AVL in practice.

Benefits of Red-Black Tree

1. **Balanced Tree** – Ensures the tree remains approximately balanced after insertions and deletions.
2. **Efficient Operations** – Guarantees $O(\log n)$ time complexity for search, insert, and delete.
3. **Fewer Rotations than AVL** – Requires fewer rotations on average compared to AVL trees.
4. **Suitable for Real-Time Systems** – Because of predictable performance, it's ideal for time-critical applications.
5. **Scalable** – Efficient even with large datasets due to guaranteed height constraints.

Applications of Red-Black Tree

1. **Used in STL (C++)** – `map`, `multimap`, `set`, and `multiset` in C++ use Red-Black Trees.
2. **Memory Management** – Used in memory allocators (like Linux Kernel's `kmalloc`).
3. **Databases & File Systems** – Helps in indexing and maintaining balanced access time.
4. **CPU Scheduling** – Linux kernel uses it in Completely Fair Scheduler.
5. **Routing Tables** – Used in networking to keep lookup tables balanced.

Features of Red-Black Tree

1. **Every Node is Red or Black** – Fundamental property of the structure.
2. **Root is Always Black** – Maintains tree balance from the top.
3. **No Two Red Nodes in a Row** – Ensures balanced paths.
4. **Same Black Height** – All paths from a node to its descendant NULL nodes contain the same number of black nodes.

5. Self-Balancing – Automatically balances itself during insertions and deletions.

Properties of Red-Black Trees

A Red-Black Tree have the following properties:

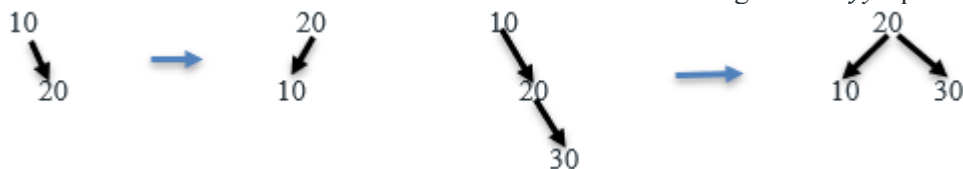
1. **Node Color:** Each node is either red or **black**.
2. **Root Property:** The root of the tree is always **black**.
3. **Red Property:** Red nodes cannot have red children (no two consecutive red nodes on any path).
4. **Black Property:** Every path from a node to its descendant null nodes (leaves) has the same number of **black** nodes.
5. **Leaf Property:** All leaves (NIL nodes) are **black**.

Rotation

Rotations are fundamental operations in maintaining the balanced structure of a Red-Black Tree (RBT). They help to preserve the properties of the tree, ensuring that the longest path from the root to any leaf is no more than twice the length of the shortest path. Rotations come in two types: **left rotations** and **right rotations**.

Left Rotation

A left rotation at node xx moves xx down to the left and its right child yy up to take xx 's place.



Before Rotation:

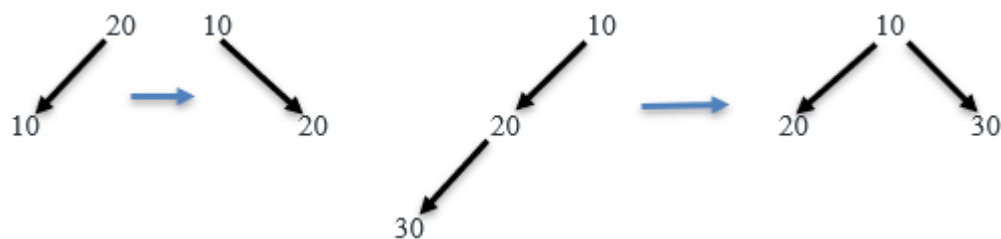
After Left Rotation:



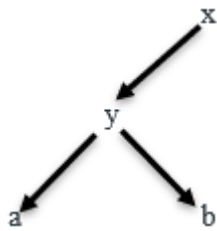
Left Rotation Steps:

1. Set y to be the right child of x .
2. Move y 's left subtree to x 's right subtree.
3. Update the parent of x and y .
4. Update x 's parent to point to y instead of x .
5. Set y 's left child to x .
6. Update x 's parent to y .

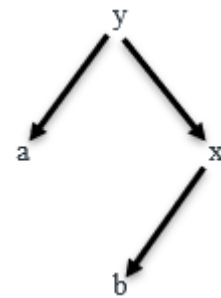
Right Rotation: A right rotation at node xx moves xx down to the right and its left child yy up to take xx 's place.



Before Right Rotation:



After Right Rotation:



Right Rotation Steps:

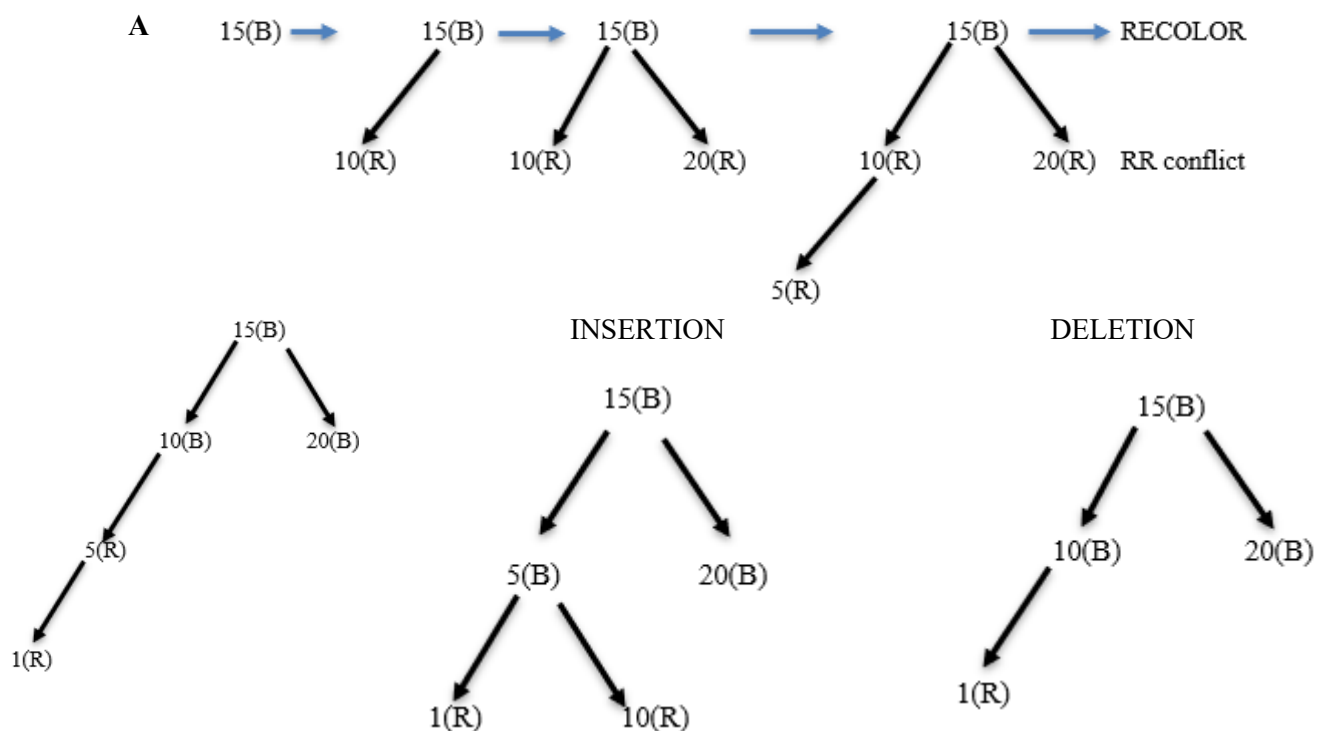
1. Set y to be the left child of x .
2. Move y 's right subtree to x 's left subtree.
3. Update the parent of x and y .
4. Update x 's parent to point to y instead of x .
5. Set y 's right child to x .
6. Update x 's parent to y .

Insertion Process in RBT

When inserting a new node:

1. Insert it like in a regular **BST** (as **Red**).
2. Then **fix violations** of Red-Black properties using:
 - **Recoloring**
 - **Rotations**: Left-Rotate and Right-Rotate.

Ques: 15, 10, 20, 5, 1



B-Tree: A B-Tree is a **multi-way self-balancing search tree**, commonly used in **databases and file systems**.

- Each node can have **more than two children** (unlike BST).
- A node can contain **multiple keys**.
- Keeps data **sorted** and allows **logarithmic time** operations.
- **Always balanced** – height is kept low even with many elements.

Benefits of B-Tree

1. **Balanced Tree:** Keeps data balanced by ensuring all leaf nodes are at the same depth.
2. **Faster Disk Access:** Optimized for systems that read/write large blocks of data (like databases).
3. **Efficient Searching:** Reduces the number of disk accesses compared to binary trees.
4. **Supports Range Queries:** Ideal for sorted data access and range-based queries.
5. **Scalable:** Performs well even with large data sizes due to its multi-level indexing.

Applications of B-Tree

1. **Databases:** Used in relational and non-relational databases for indexing (e.g., MySQL, SQLite).
2. **File Systems:** Common in file systems like NTFS and HFS+ for directory and file indexing.
3. **Operating Systems:** Used in memory management and virtual memory indexing.
4. **Search Engines:** For indexing large sets of words or documents.
5. **Cloud Storage:** Helps manage large distributed datasets efficiently.

Features of B-Tree

1. **Multi-way Tree:** Each node can have more than two children (unlike binary trees).
2. **Sorted Keys:** All keys in a node are stored in sorted order for efficient traversal.
3. **Height Balanced:** Guarantees logarithmic height, ensuring fast operations.
4. **Dynamic Growth:** Automatically adjusts its height and node contents as data is added or removed.
5. **Internal & Leaf Nodes:** Internal nodes guide search; data may be stored in leaf or all nodes.

Insertion in a B-Tree:

1. **Find the correct leaf** node where the key should be inserted.
2. Insert the key into the leaf node in **sorted order**.
3. If the node becomes **overfull** (has m keys), **split** it:
 - Move the middle key to the parent node.
 - Split the node into two.
4. If the parent is also full, recursively split up to the root.
5. If the root splits, the tree's **height increases**.

Example: B-Tree of order 3 (max 2 keys per node)

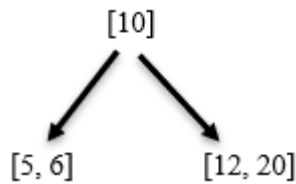
- Insert: 10, 20, 5, 6, 12

Step-by-step:

Insert 10 → [10]

Insert 20 → [10, 20]

Insert 5 → [5, 10, 20] → Overflow → Split to:



Insert 6 → [5, 6]

Insert 12 → Goes to right node [20] → becomes [12, 20]

Splay Tree – A Self-Balancing Binary Search Tree: A **Splay Tree** is a type of **self-balancing Binary Search Tree (BST)** where recently accessed elements are **moved to the root** using a process called "**splaying**". This improves the access time for elements that are frequently used. Splay tree is a self-adjusting binary search tree data structure, which means that the tree structure is adjusted dynamically based on the accessed or inserted elements. In other words, the tree automatically reorganizes itself so that frequently accessed or inserted elements become closer to the root node.

Benefits of Splay Tree

1. **Self-adjusting:** Recently accessed elements are moved to the root, improving access time for frequently used elements.
2. **Amortized Efficiency:** Guarantees $O(\log n)$ time complexity on average (amortized) for operations like insert, delete, and search.
3. **No Extra Storage:** Doesn't require additional memory for balancing (unlike AVL or Red-Black Trees).
4. **Improves Locality:** Helps improve data access patterns where recently accessed elements are likely to be accessed again.
5. **Simpler Implementation:** Conceptually simpler than AVL or Red-Black Trees, although the rotations are more frequent.

Applications of Splay Tree

1. **Memory Management:** Used in memory allocators and caching where recent data access is common.
2. **Garbage Collection:** Useful in certain garbage collection algorithms to manage memory blocks.
3. **Data Compression:** Employed in splay-based versions of **move-to-front** algorithms.
4. **Cache Systems:** Used in systems where frequently accessed data should be retrieved quickly.
5. **Network Routing Tables:** Helps maintain frequently accessed routing paths efficiently.

Features of Splay Tree

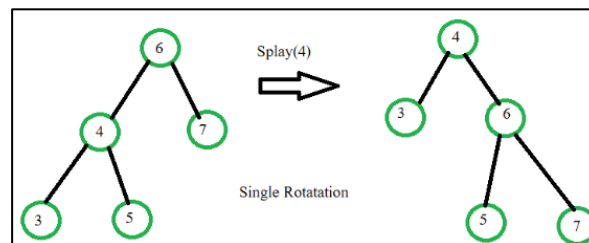
1. **Self-balancing:** Automatically adjusts its structure during insert, delete, and search operations.
2. **Uses Rotations:** Performs **splaying (rotations)** to bring accessed node to the root.

3. **Adaptive:** Performs better if access pattern is **non-uniform** (e.g., same elements accessed repeatedly).
4. **Binary Search Tree Property:** Maintains the standard BST invariant (left < root < right).
5. **No Strict Balancing:** Unlike AVL/Red-Black Trees, it doesn't maintain a strict height balance.

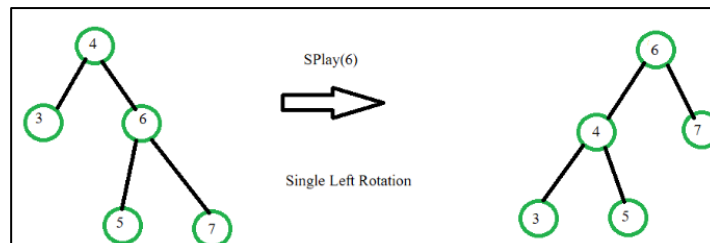
To bring a node to the root, the tree uses **rotations**, based on the node's relationship with its parent and grandparent:

- Zig Rotation
- Zag Rotation
- Zig - Zig Rotation
- Zag - Zag Rotation
- Zig - Zag Rotation
- Zag - Zig Rotation

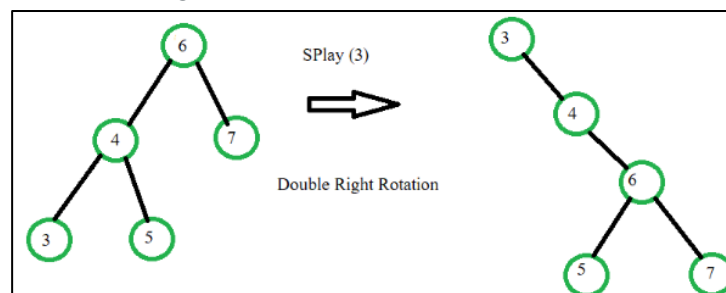
1) Zig Rotation: The Zig Rotation in splay trees operates in a manner similar to the single right rotation in AVL Tree rotations. This rotation results in nodes moving one position to the right from their current location. For example, consider the following scenario:



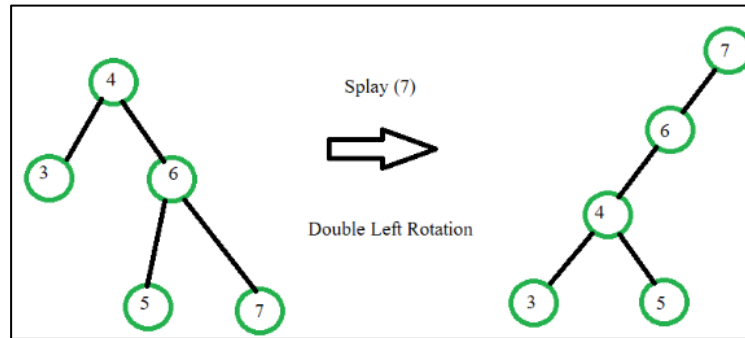
Zag Rotation: The Zag Rotation in splay trees operates in a similar fashion to the single left rotation in AVL Tree rotations. During this rotation, nodes shift one position to the left from their current location. For instance, consider the following illustration:



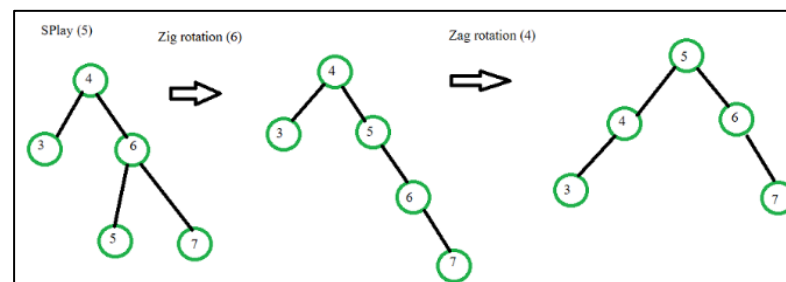
Zig-Zig Rotation: The Zig-Zig Rotation in splay trees is a double zig rotation. This rotation results in nodes shifting two positions to the right from their current location. Take a look at the following example for a better understanding:



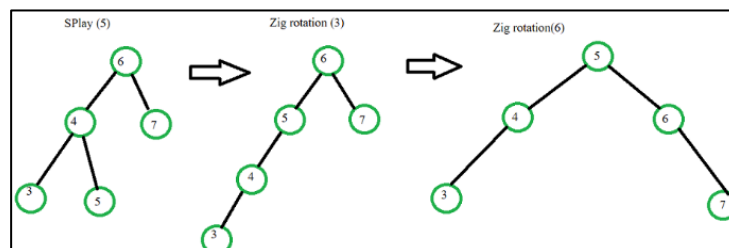
Zag-Zag Rotation: In splay trees, the Zag-Zag Rotation is a double zag rotation. This rotation causes nodes to move two positions to the left from their present position. For example:



Zig-Zag Rotation: The Zig-Zag Rotation in splay trees is a combination of a zig rotation followed by a zag rotation. As a result of this rotation, nodes shift one position to the right and then one position to the left from their current location. The following illustration provides a visual representation of this concept:



Zag-Zig Rotation: The Zag-Zig Rotation in splay trees is a series of zag rotations followed by a zig rotation. This results in nodes moving one position to the left, followed by a shift one position to the right from their current location. The following illustration offers a visual representation of this concept:



Applications of the splay tree:

- **Caching:** Splay trees can be used to implement cache memory management, where the most frequently accessed items are moved to the top of the tree for quicker access.
- **Database Indexing:** Splay trees can be used to index databases for faster searching and retrieval of data.
- **File Systems:** Splay trees can be used to store file system metadata, such as the allocation table, directory structure, and file attributes.
- **Data Compression:** Splay trees can be used to compress data by identifying and encoding repeating patterns.
- **Text Processing:** Splay trees can be used in text processing applications, such as spell-checkers, where words are stored in a splay tree for quick searching and retrieval.
- **Graph Algorithms:** Splay trees can be used to implement graph algorithms, such as finding the shortest path in a weighted graph.
- **Online Gaming:** Splay trees can be used in online gaming to store and manage high scores, leaderboards, and player statistics.

Benefits of Splay Tree

1. **Self-adjusting:** Recently accessed elements are moved to the root, improving access time for frequently used elements.
2. **Amortized Efficiency:** Guarantees $O(\log n)$ time complexity on average (amortized) for operations like insert, delete, and search.
3. **No Extra Storage:** Doesn't require additional memory for balancing (unlike AVL or Red-Black Trees).
4. **Improves Locality:** Helps improve data access patterns where recently accessed elements are likely to be accessed again.
5. **Simpler Implementation:** Conceptually simpler than AVL or Red-Black Trees, although the rotations are more frequent.

Advantages of Splay Trees:

Splay trees have amortized time complexity of $O(\log n)$ for many operations, making them faster than many other balanced tree data structures in some cases.

Splay trees are self-adjusting, meaning that they automatically balance themselves as items are inserted and removed. This can help to avoid the performance degradation that can occur when a tree becomes unbalanced.

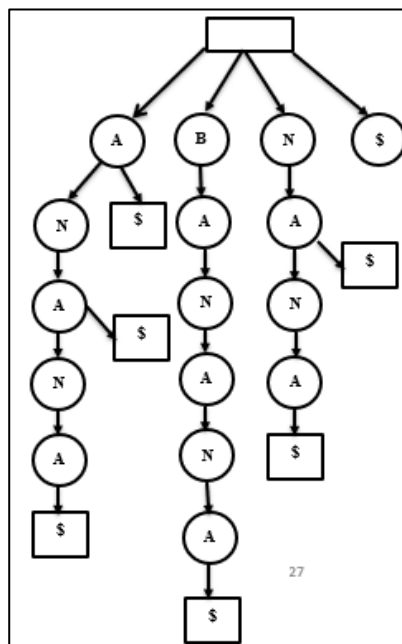
Disadvantages of Splay Trees:

Splay trees can have worst-case time complexity of $O(n)$ for some operations, making them less predictable than other balanced tree data structures like AVL trees or red-black trees.

Splay trees may not be suitable for certain applications where predictable performance is required.

Suffix Tree in Data Structures: A Suffix Tree is a special kind of compressed trie (prefix tree) used for string processing. It represents all the suffixes of a given string.

- Every **suffix** of the string is stored as a **path** from the root to a leaf.
- Built in **O(n)** time for a string of length n (using Ukkonen's algorithm).
- Used for fast **pattern matching**, **substring search**, **longest repeated substring**, etc.



AVL Tree Data Structure

An **AVL tree** defined as a self-balancing Binary Search Tree (BST) where the difference between heights of left and right subtrees for any node cannot be more than one.

- The absolute difference between the heights of the left subtree and the right subtree for any node is known as the **balance factor** of the node. The balance factor for all nodes must be less than or equal to 1.
- Every AVL tree is also a Binary Search Tree (Left subtree values Smaller and Right subtree values greater for every node), but every BST is not AVL Tree. For example, the second diagram below is not an AVL Tree.
- The main advantage of an AVL Tree is, the time complexities of all operations (search, insert and delete, max, min, floor and ceiling) become $O(\log n)$. This happens because height of an AVL tree is bounded by $O(\log n)$. In case of a normal BST, the height can go up to $O(n)$.
- An AVL tree maintains its height by doing some extra work during insert and delete operations. It mainly uses rotations to maintain both BST properties and height balance.
- There exist other self-balancing BSTs also like Red Black Tree. Red Black tree is more complex, but used more in practice as it is less restrictive in terms of left and right subtree height differences.
- **Example of an AVL Tree:**
- The balance factors for different nodes are : 12 :1, 8:1, 18:1, 5:1, 11:0, 17:0 and 4:0. Since all differences are less than or equal to 1, the tree is an AVL tree.

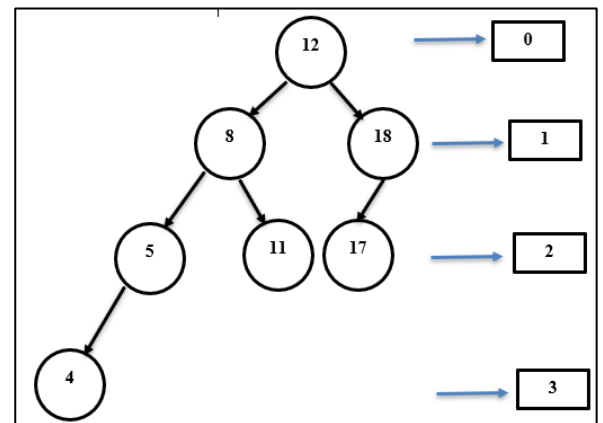
An AVL Data Structure is a self balancing Binary Search Tree. That maintain balance to ensure efficient operations.

An AVL tree is a type of Binary Search Tree used in computer science. In an AVL each node has a balance factor(BF). The balance factor is the difference between the left and right sub trees. $BF = 0, 1, -1$

LST-RST

$2-2=0$

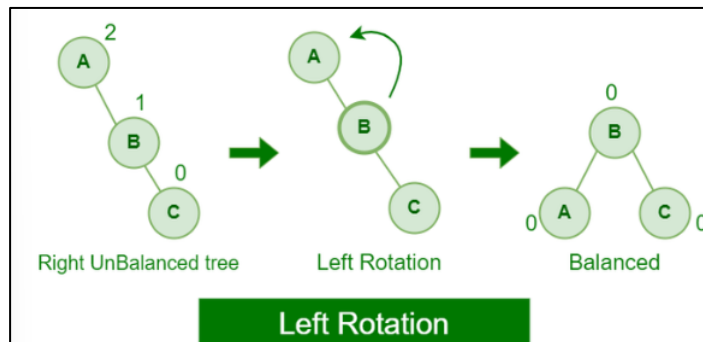
Balance factor can only be -1, 0, 1. If the balance factor gets outside this range the tree does a rotation to fix itself.



Property	Description
Balance Factor	For every node, Balance Factor = Height(left subtree) - Height(right subtree) must be -1, 0, or +1 .
Binary Search Tree	Binary Search Tree
Self-Balancing	Automatically re-balances itself after insertions and deletions using rotations.
Rotations	Four types of rotations used to maintain balance: ▪ Right Rotation (LL) ▪ Left Rotation (RR) ▪ Left-Right Rotation (LR) ▪ Right-Left Rotation (RL)
Time Complexity	All operations (search, insert, delete) take $O(\log n)$ time because the tree remains balanced.
Height Limit	The height of an AVL tree with n nodes is always $\leq 1.44 \times \log_2(n + 2) - 0.328$.
No Duplicate Values	Typically, AVL trees do not allow duplicate keys.

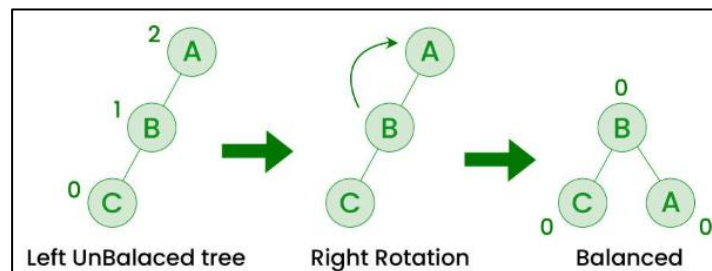
Rotating the subtrees (Used in Insertion and Deletion): An AVL tree may rotate in one of the following four ways to keep itself balanced while making sure that the BST properties are maintained.

Left Rotation: When a node is added into the right subtree of the right subtree, if the tree gets out of balance, we do a single left rotation.



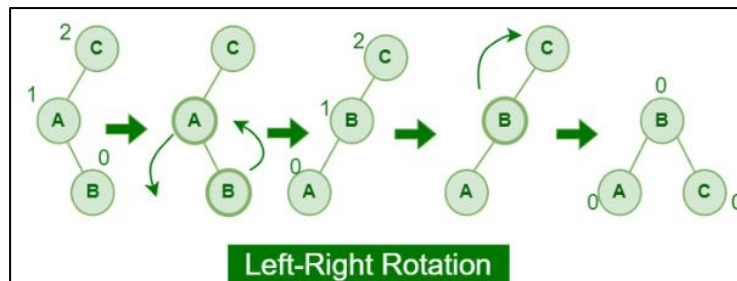
Right Rotation:

If a node is added to the left subtree of the left subtree, the AVL tree may get out of balance, we do a single right rotation.



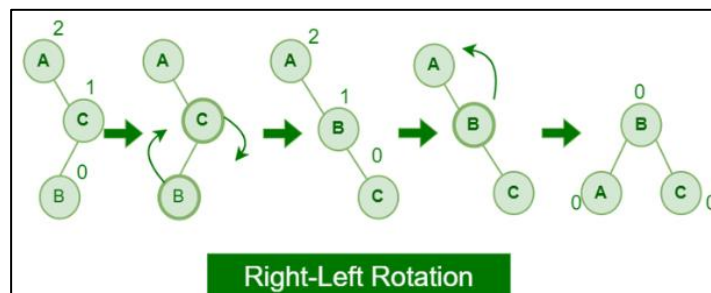
Left-Right Rotation:

A left-right rotation is a combination in which first left rotation takes place after that right rotation executes.



Right-Left Rotation:

A right-left rotation is a combination in which first right rotation takes place after that left rotation executes.



DFS (Depth First Search) in Data Structure

DFS (Depth First Search) is a fundamental **graph traversal algorithm** used to explore nodes and edges of a graph or tree **by going as deep as possible along one path** before backtracking.

Features of DFS:

- **Starts at a source node** and explores as far as possible along each branch before backtracking.
- Can be implemented **recursively** or using a **stack**.
- Used in **graph traversal, topological sorting, cycle detection, maze solving**, etc.

Applications of DFS:

- Detecting **cycles** in graphs.
- **Topological sorting** in DAGs.
- **Pathfinding** in mazes and puzzles.
- **Component detection** in undirected graphs.

Minimum Spanning Tree (MST)

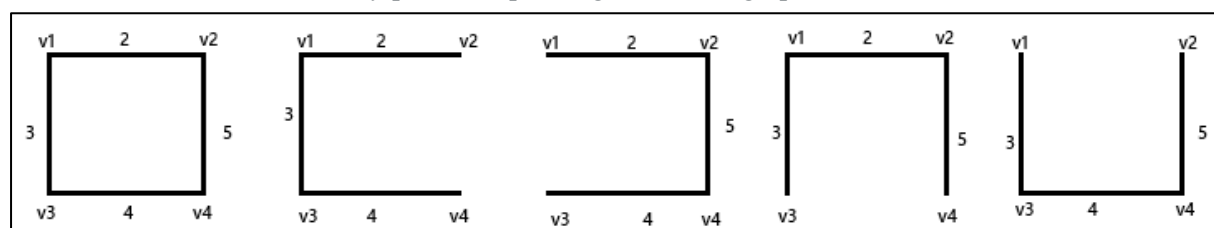
A **spanning tree** is defined as a tree-like subgraph of a connected, undirected graph that includes all the vertices of the graph. Or, to say in Layman's words, it is a subset of the edges of the graph that forms a tree (**acyclic**) where every node of the graph is a part of the tree.

- The minimum spanning tree has all the properties of a spanning tree with an added constraint of having the minimum possible weights among all possible spanning trees. Like a spanning tree, there can also be many possible MSTs for a graph.

Properties of a Spanning Tree

The spanning tree holds the **below-mentioned principles**:

- The number of vertices (**V**) in the graph and the spanning tree is the same.
- There is a fixed number of edges in the spanning tree which is equal to one less than the total number of vertices ($E = V - 1$).
- The spanning tree should not be **disconnected**, as in there should only be a single source of component, not more than that.
- The spanning tree should be **acyclic**, which means there would not be any cycle in the tree.
- The total cost (or weight) of the spanning tree is defined as the sum of the edge weights of all the edges of the spanning tree.
- There can be many possible spanning trees for a graph.



Minimum spanning tree have two techniques are as follows:

(a) Kruskal algorithm

(b) Prims algorithm

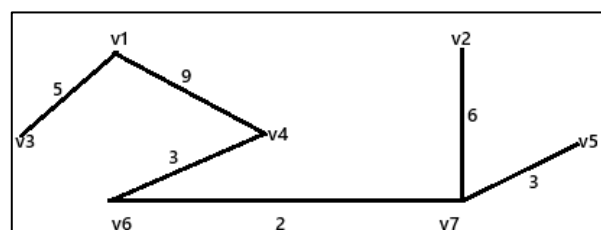
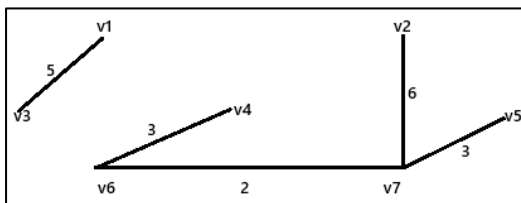
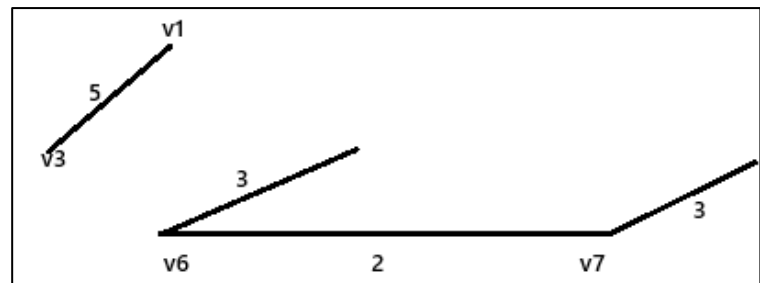
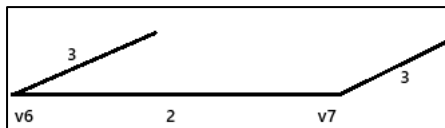
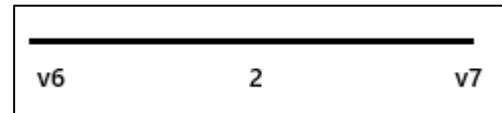
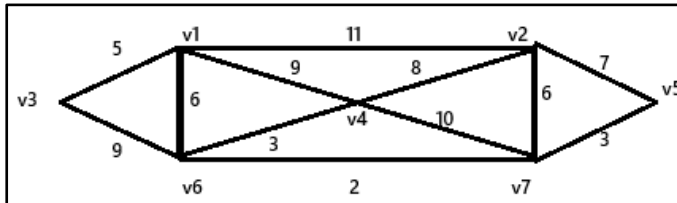
A minimum spanning tree (MST) or minimum weight spanning tree for a weighted, connected, and undirected graph is a **spanning tree** (no cycles and connects all vertices) that has minimum weight. The weight of a spanning tree is the sum of all edges in the tree.

In Kruskal's algorithm, we sort all edges of the given graph in increasing order. Then it keeps on adding new edges and nodes in the MST if the newly added edge does not form a cycle. It picks the minimum weighted edge at first and the maximum weighted edge at last. Thus we can say that it makes a locally optimal choice in each step in order to find the optimal solution.

How to find MST using Kruskal's algorithm?

Below are the steps for finding MST using Kruskal's algorithm:

- Sort all the edges in a non-decreasing order of their weight.
- Pick the smallest edge. Check if it forms a cycle with the spanning tree formed so far. If the cycle is not formed, include this edge. Else, discard it.
- Repeat step 2 until there are $(V-1)$ edges in the spanning tree.



Prim's Algorithm for Minimum Spanning Tree (MST)

Prim's algorithm is a Greedy algorithm like [Kruskal's algorithm](#). This algorithm always starts with a single node and moves through several adjacent nodes, in order to explore all of the connected edges along the way.

- The algorithm starts with an empty spanning tree.
- The idea is to maintain two sets of vertices. The first set contains the vertices already included in the MST, and the other set contains the vertices not yet included.

- At every step, it considers all the edges that connect the two sets and picks the minimum weight edge from these edges. After picking the edge, it moves the other endpoint of the edge to the set containing MST.

A group of edges that connects two sets of vertices in a graph is called Articulation Points (or Cut Vertices) in graph theory. So, at every step of Prim's algorithm, find a cut, pick the minimum weight edge from the cut, and include this vertex in MST Set (the set that contains already included vertices).

Working of the Prim's Algorithm

Step 1: Determine an arbitrary vertex as the starting vertex of the MST. We pick 0 in the below diagram.

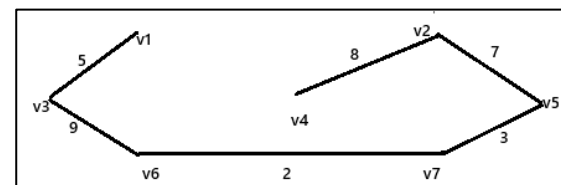
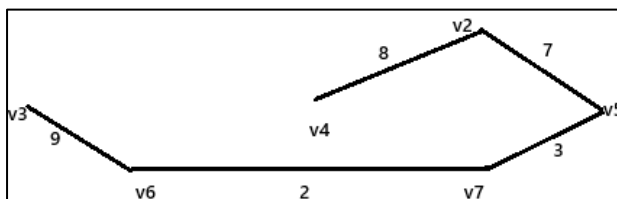
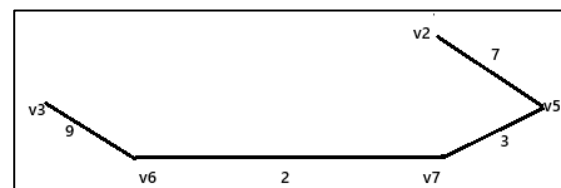
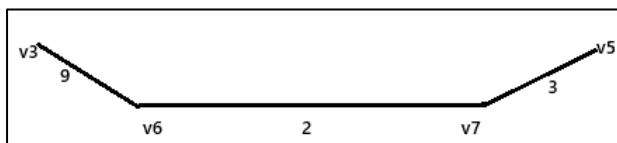
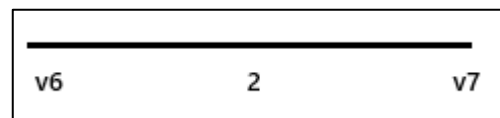
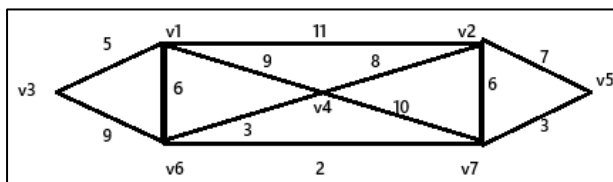
Step 2: Follow steps 3 to 5 till there are vertices that are not included in the MST (known as fringe vertex).

Step 3: Find edges connecting any tree vertex with the fringe vertices.

Step 4: Find the minimum among these edges.

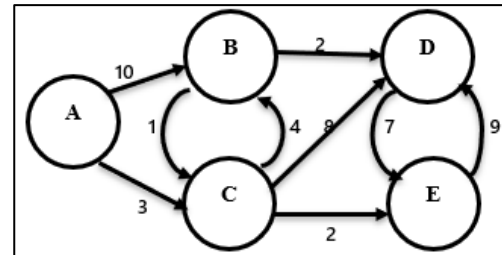
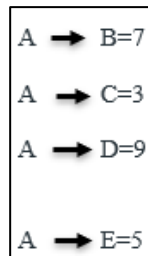
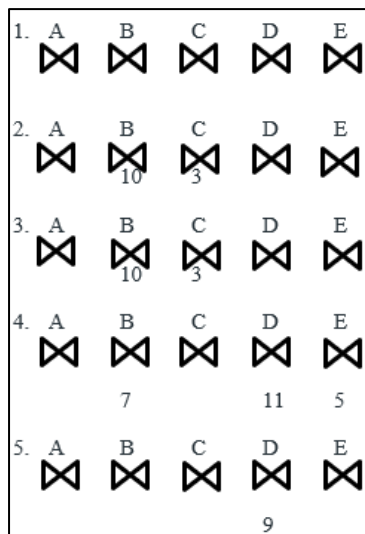
Step 5: Add the chosen edge to the MST. Since we consider only the edges that connect fringe vertices with the rest, we never get a cycle.

Step 6: Return the MST and exit.



The algorithm maintains a set of visited vertices and a set of unvisited vertices. It starts at the source vertex and iteratively selects the unvisited vertex with the smallest tentative distance from the source. It then visits the neighbors of this vertex and updates their tentative distances if a shorter path is found. This process continues until the destination vertex is reached, or all reachable vertices have been visited. It is an algorithm that solves the single source shortest path problem for a

directed graph with non negative edges. It maintains a set of vertices where final shortest path waits for the source have already been determined.

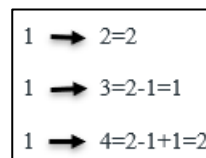
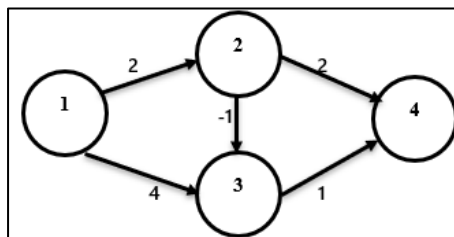


Bellman-Ford Algorithm in Data Structure

The **Bellman-Ford Algorithm** is a **shortest path algorithm** used to find the **minimum distance** from a **single source vertex** to **all other vertices** in a **weighted graph**, including graphs with **negative weights**.

Key Features:

- Works with **negative edge weights** (unlike Dijkstra).
- Can **detect negative weight cycles**.



Floyd-Warshall Algorithm in Data Structures

The **Floyd-Warshall algorithm** is a **dynamic programming** technique used to **find the shortest paths between all pairs of vertices** in a weighted graph. It's especially useful for **dense graphs** and graphs represented using an **adjacency matrix**.

Key Points

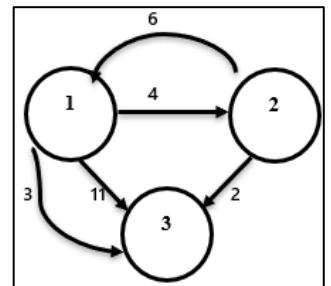
- Works for **directed or undirected** graphs.
- Handles **positive and negative edge weights**, but **no negative weight cycles**.

A0	1	2	3
1	0	4	11
2	6	0	2
3	3	-	0

A1	1	2	3
1	0	4	11
2	6	0	2
3	3	7	0

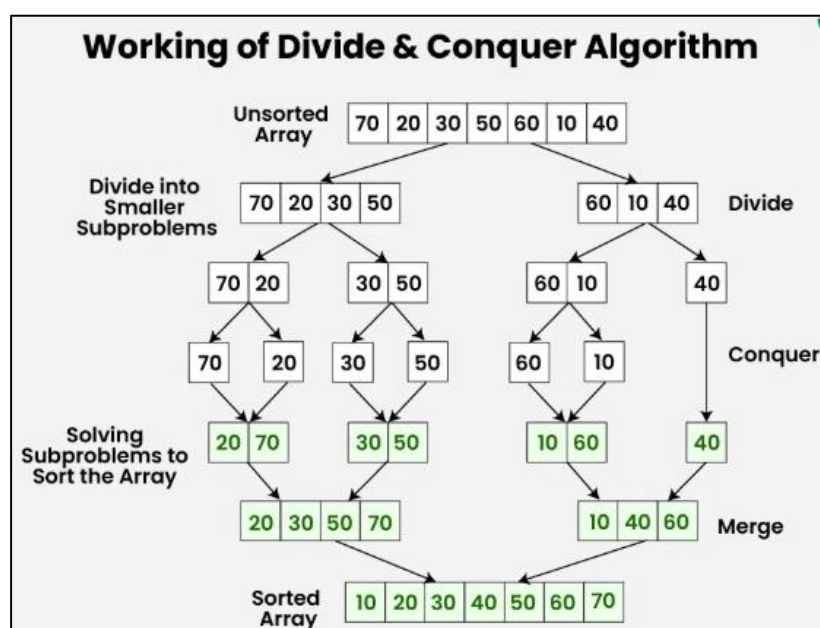
A2	1	2	3
1	0	4	6
2	6	0	2
3	3	7	0

A1	1	2	3
1	0	4	6
2	5	0	2
3	3	7	0



Algorithm design and techniques

1. **Introduction to Divide and Conquer Algorithm:** Divide and Conquer Algorithm is a problem-solving technique used to solve problems by dividing the main problem into subproblems, solving them individually and then merging them to find solution to the original problem. Divide and Conquer is mainly useful when we divide a problem into independent subproblems. If we have overlapping subproblems, then we use [Dynamic Programming](#). Working of Divide and Conquer Algorithm. Divide and Conquer Algorithm can be divided into three steps: **Divide**, **Conquer** and **Merge**.



2. **Greedy Algorithm in Data Structure:** A greedy algorithm is a problem-solving strategy that makes the **locally optimal choice at each step** with the hope of finding a **global optimum**. A greedy algorithm builds up a solution piece by piece, **always choosing the next piece that offers the most immediate benefit (greedy choice)**. Greedy is an algorithmic paradigm that builds up a solution **piece by piece**, always choosing the next piece that offers the most **obvious and immediate benefit**. Greedy algorithms are used for optimization problems. This is an algorithm design technique in which most of the problems solved have N inputs and require us to obtain a subset that satisfies some constraints. Any subset that satisfies these constraints is called a feasible solution. We need to find a feasible that eighteen maximizes or minimises a given objective function A feasible solution that does this is called an optimal solution. The steps for achieving greedy algorithm are:

(a) Feasible

(b) Optimal choice

(c) unalterable

Example Important: Knapsack Problem using Greedy Algorithm in Data Structure

The **Knapsack Problem** is a classic optimization problem. In **Greedy Algorithm**, we typically solve the **Fractional Knapsack Problem**. In this we have to pack the knapsack with maximum value in such a manner that the total weight of the items should not be greater than the capacity knapsack. There we are presenting the greedy strategy where the items are arranged in sum order which is based on the following criteria:

Greedy criteria 1: In this the items are arranged by their value. The item with maximum value is selected first and the process continues till the minimum value.

Greedy criteria 2: In this items are arranged by their weights. The item with lightest weight is selected first and the process continues till maximum weight.

Greedy criteria 3: In this the items are arranged by certain ratio R where $R = \text{value} / \text{Weight}$. There selection proceeds from maximum ratio to minimum ratio.

Ques: $n=3, w=25$

$(v_1, v_2, v_3) = (30, 29, 20)$

$(w_1, w_2, w_3) = (23, 20, 15)$

I. Max value first= $v_1=30, w_1=23$

Left= $25-23=2$

$V_2=29, w=20$

Fraction=left weight/ Weight *value = $2/ 20 * 29 = 2.9$

Total profit= v_1 +fraction = Total profit= $30+2.9=32.9$

2. Min weight first= $v_3=20, w_3=15$

Left= $25-15=10$

$V_2= 29, w_2= 20$

Fraction= $10/20 * 29 = 14.5$

Profit= $20 + 14.5 = 34.5$

3. $R1 = 30/23 = 1.34$
 $R2 = 29/20 = 1.4$
 $R3 = 20/15 = 1.33$
 $V2 = 29, w2 = 20$
 $left = 25 - 20 = 5$
 $v1 = 30, w1 = 23$
 $fraction = 5/23 * 30 = 6.5$
 $profit = 29 + 6.5 = 35.5$
Profit=

Case	Feasible
I.	32.9
II.	34.5
III.	35.5(optimal)

Fractional Knapsack Problem (Greedy Approach)

Problem Statement:

- You are given n items, each with a **weight** and a **value**.
- A **knapsack** can carry a maximum weight W .
- **Goal:** Maximize total value by choosing whole or fractional items.

Greedy Strategy:

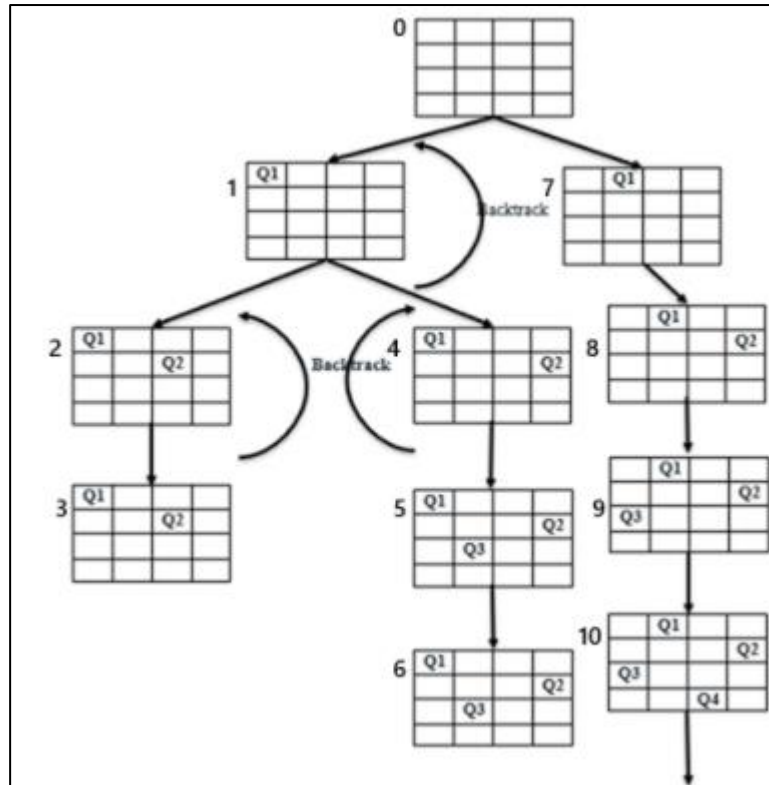
- Calculate **value/weight** ratio for each item.
- Sort items in **descending order of value/weight**.
- Pick items starting from the highest ratio:
 - If the full item fits, take it.
 - Otherwise, take the fraction that fits.

Backtracking in Data Structures: **Backtracking** is a **problem-solving algorithmic technique** used to build solutions **incrementally** and **abandon** them ("backtrack") as soon as it becomes clear that they cannot lead to a valid solution. Backtracking is like trying different paths, and when you hit a dead end, you backtrack to the last choice and try a different route. In this article, we'll explore the basics of backtracking, how it works, and how it can help solve all sorts of challenging problems. It's like a method for finding the right way through a complex choices. The main point for backtracking algorithm is the binary choice "no" it means the algorithm has encountered deadend and it backtracks one step and will try a different path for choice "yes".

Example N queen problem(4 and 8 queen):

- You are given an $N \times N$ **chessboard**.
- The goal is to **place N queens** on the board such that **no two queens attack each other**.
- A queen can attack any piece in:
 - The same **row**
 - The same **column**
 - Any **diagonal**

- Given an integer **n**, the task is to find the solution to the **n-queens problem**, where **n** queens are placed on an **n*n** chessboard such that no two queens can attack each other.
- The N Queen is the problem of placing N chess queens on an N×N chessboard so that no two queens attack each other.
- The problem is to place N Queen's in such a manner on N*N chessboard such that number 2 Queens attack each other by being in the same row, column and diagonal.



Solution:<4, 6, 8, 2, 7, 1, 3, 5>

Solution:<2, 4, 1,3>

			Q1				
					Q2		
							Q3
	Q4						
						Q5	
Q6							
		Q7					
				Q8			

Branch and Bound", which is a popular **algorithm design technique** used in **data structures and algorithms**, especially for solving **combinatorial optimization problems** like:

- **Travelling Salesman Problem (TSP)**
- **0/1 Knapsack Problem**
- **Job Scheduling**
- **Shortest Path** (in special cases)

Branch and Bound is a **systematic search algorithm** used to find the **optimal solution** by exploring all possible options in a smart way, **eliminating (bounding) paths** that cannot lead to a better result.

In this technique at each stage we calculate the bound for a particular node and check whether this bound will be able to give the solution or not. It means we calculate how far we are from the solution. In this we have two requirements **(a)** Lower bound and upper bound **(b)** Pervious checking. **Example:** Traveling sales problem(TSP): This is a problem a salesman needs to visit N cities in such a manner that all the cities must be visited at once and in the end he returns the city from where he started with minimum cost. The TSP is to found a route starting and ending at same node that will take in all cities with minimum cost.

	A	B	C	D
A	0	10	15	20
B	10	0	35	25
C	15	35	0	30
D	20	25	30	0

Cost Adjacency Matrix

Shortest path A $\rightarrow$ B $\rightarrow$ D $\rightarrow$ C $\rightarrow$ A = 80

